

Raport badawczy: klasyfikacja grzybów jadalnych i trujących przy użyciu metod Uczenia Maszynowego

1. Analiza i Eksploracja Bazy Danych

Celem projektu jest zbudowanie systemu sztucznej inteligencji zdolnego do precyzyjnej klasyfikacji grzybów na podstawie ich cech morfologicznych na dwie klasy: jadalne (**e** – *edible*) oraz trujące (**p** – *poisonous*).

Analizę oparto na bazie danych *Mushroom Classification Dataset* pochodzący z repozytorium UCI Machine Learning. Zbiór ten zawiera szczegółowe opisy właściwości morfologicznych (wizualnych i zapachowych) dziko rosnących grzybów. Zadaniem modeli AI było wypracowanie takich reguł, aby na podstawie samych cech zewnętrznych bezbłędnie ocenić bezpieczeństwo okazu.

1.1. Analiza struktury i zbalansowania zbioru danych

Baza danych składa się z **8124 rekordów** (wierszy) opisujących unikalne okazy grzybów. Każdy obiekt scharakteryzowany jest przez **22 cechy wejściowe** (kolumny), takie jak kształt kapelusza, jego powierzchnia, kolor, obecność pierścienia czy zapach. Kluczowym elementem oceny przydatności zbioru do celów badawczych było sprawdzenie zbalansowania klas decyzyjnych. Wykorzystanie funkcji zliczającej unikalne wartości przyniosło następujące rezultaty:

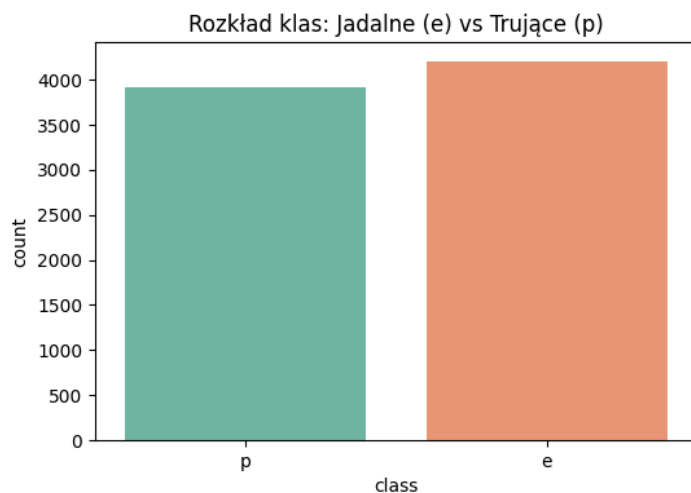
```
print(df['class'].value_counts())
```

Wynik z terminala:

```
Liczba grzybów jadalnych (e) i trujących (p):  
class  
e      4208  
p      3916  
Name: count, dtype: int64
```

Zbiór charakteryzuje się zbalansowaniem klas (około 51.8% jadalnych do 48.2% trujących). Oznacza to, że nie ma potrzeby stosowania zaawansowanych technik resamplingu, ponieważ żadna z klas nie jest mniejszościowa.

W celu wizualizacji dystrybucji danych wygenerowano wykres słupkowy (rozkład klas) za pomocą bibliotek matplotlib i seaborn:



2. Przygotowanie danych do modelowania (Preprocessing)

2.1. Weryfikacja kompletności danych

Przed uruchomieniem algorytmów dane musiały zostać poddane odpowiednim transformacjom. Jednym z pierwszych etapów przygotowania danych jest identyfikacja i sprawdzenie, czy w bazie nie ma brakujących wartości. Sprawdzenia tego dokonano za pomocą następującej instrukcji:

```
print(df.isnull().sum().sum())
```

Terminal zwrócił wynik 0, co dowodzi, że baza jest kompletna i nie wymaga sztucznego uzupełniania braków.

2.2. Transformacja zmiennych (Encoding)

Kolejnym krokiem była transformacja formatu danych. Wszystkie cechy w oryginalnym pliku zapisane były literami. Modele matematyczne oraz sztuczne sieci neuronowe nie potrafią jednak kalkulować odległości ani modyfikować wag dla znaków tekstowych – wymagają liczb. Zastosowano więc dwie operacje:

1. Mapowanie: Klasę celu zamieniono na format binarny, gdzie grzyb jadalny otrzymał etykietę 0, a trujący 1.
2. One-Hot Encoding: Funkcja `pd.get_dummies` przekształciła tekstowe wartości cech na niezależne kolumny numeryczne o wartościach 0 lub 1. Dzięki temu tekst stał się zapisem zero-jedynkowym.

2.3. Podział danych na zbiór treningowy i testowy

Ostatnim etapem przygotowania było rozbiecie danych funkcją `train_test_split` na zbiór treningowy (80% danych) i testowy (20% danych – dokładnie 1625 obiektów). Parametr `random_state=42` zagwarantował stałe ziarno losowości, co zapewnia pełną powtarzalność eksperymentu przy każdym kolejnym uruchomieniu programu.

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

Wielkość próby testowej można dynamicznie kontrolować poprzez właściwość `X_test.shape[0]`, która zwraca liczbę wierszy wyodrębnionych do testu (wynosi ona dokładnie 1625 obiektów).

3. Klasyfikacja i analiza wyników

W ramach eksperymentu przetestowano cztery zróżnicowane algorytmy sztucznej inteligencji. Do ich oceny wykorzystano raporty tekstowe oraz macierze pomyłek, które pokazują dokładną liczbę sukcesów i błędów modelu.

3.1. Architektura Funkcji Ewaluacyjnej

Stworzono funkcję `uruchom_i_ocen_model`, która **skupia w sobie i automatyzuje** cały proces uczenia, testowania oraz wyliczania statystyk końcowych, dzięki czemu nie trzeba pisać tego samego kodu oddzielnie dla każdego modelu.

```
def uruchom_i_ocen_model(nazwa_modelu, model):
    model.fit(X_train, y_train)      # NAUKA: Model analizuje dane treningowe
    y_pred = model.predict(X_test)   # TEST: Model próbuje zgadnąć status nowych grzybów

    acc = accuracy_score(y_test, y_pred) # Liczymy ogólną dokładność
    results[nazwa_modelu] = acc * 100

    print(f"\n{nazwa_modelu}")
    print(f"Dokładność ogólna: {acc*100:.2f}%") #acc - ogólna dokładność modelu zapisana jako ułamek, :.2f - 2 liczby po przecinku
    print("\nRaport szczegółowy (Precision, Recall):")
    print(classification_report(y_test, y_pred, target_names=['Jadalny', 'Trujący']))
    print("Macierz pomyłek (Confusion Matrix):")
    print(confusion_matrix(y_test, y_pred))
```

3.2 Wyniki przetestowanych modeli

1. Drzewo Decyzyjne (Decision Tree) – badanie głębokości

W tym punkcie sprawdzono, jak zmiana dopuszczalnej liczby pytań zadawanych przez drzewo wpływa na jakość wyniku.

- Głębokość skrajnie płytka ($\text{max_depth}=1$): Model osiągnął słabe 85% dokładności. Z analizy macierzy pomyłek wynika, że model dopuścił się aż 241 błędów typu False Negative – czyli zaklasyfikował trujące grzyby jako jadalne. Świadczy to o zjawisku niedouczenia (underfittingu) z powodu zbyt prostych reguł.
- Głębokość optymalna ($\text{max_depth}=5$): Po zwiększeniu limitu pytań, drzewo wygenerowało perfekcyjną siatkę logiczną, uzyskując 100% dokładności i zero błędów.

2. K-Najbliższych Sąsiadów (KNN)

Model oparty na szukaniu podobnych obiektów geometrycznych osiągnął wynik **100%**. Świadczy to o tym, że po zamianie tekstu na liczby, grzyby jadalne i trujące tworzą bardzo wyraźnie oddzielone od siebie grupy.

3. Naiwny Klasyfikator Bayesowski (Naive Bayes)

Algorytm ten zanotował najslabszy rezultat wśród w pełni nauczonych modeli, uzyskując 93.60% dokładności i popełniając 52 krytyczne błędy (przepuszczenie trującego grzyba).

- Wniosek: Naiwny Bayes zakłada całkowitą niezależność cech od siebie. W biologii cechy grzybów (zapach, struktura, kolor) są ze sobą powiązane. To teoretyczne uproszczenie matematyczne stało się przyczyną niższej sprawności tego modelu.

4. Sztuczna Sieć Neuronowa (MLP Classifier)

Zaimplementowano wielowarstwowy **model sieci neuronowej** z jedną warstwą ukrytą składającą się z 10 sztucznych neuronów. Sieć bezbłędnie połączyła zależności w danych, osiągając wynik **100% poprawnych odpowiedzi**. Potwierdza to, że nawet prosta i lekka struktura sieciowa idealnie rozwiązuje ten problem.

4. Poszukiwanie ukrytych korelacji (Reguły asocjacyjne)

Podjęto próbę znalezienia bezpośrednich, silnych zależności pomiędzy pojedynczymi atrybutami a toksycznością. Wykorzystano do tego tablicę krzyżową (pd.crosstab) z normalizacją procentową dla cechy zapachu (odor):

```
tabela_zaleznosci = pd.crosstab(df['odor'], df['class'], normalize='index') * 100
```

Wynik analizy: Eksperyment ujawnił obecność bezwzględnych reguł deterministycznych. Przykładowo, zapach oznaczony jako f (nieprzyjemny) w 100% przypadków asocjuje z grzybem trującym. Z kolei zapach a (anyżowy) w 100% cechuje wyłącznie okazy jadalne. Istnienie tak silnych, pojedynczych korelacji wyjaśnia, dlaczego modele takie jak Drzewo Decyzyjne czy KNN tak szybko osiągnęły stuprocentową skuteczność.

5. Podsumowanie i Wnioski Końcowe

1. Przetestowano zarówno klasyczne podejście statystyczne (Bayes, Drzewa, KNN), jak i zaawansowane modelowanie za pomocą sieci neuronowych.
2. Udowodniono w praktyce (empirycznie), że właściwy dobór parametrów (np. odpowiednia głębokość drzewa) ma kluczowe znaczenie dla bezpieczeństwa działania systemów klasyfikacji.
3. Wizualne podsumowanie pracy stanowi drugi wygenerowany wykres, prezentujący zestawienie dokładności wszystkich struktur. Jasno pokazuje on przewagę modeli nieliniowych i geometrycznych nad uproszczonym podejściem probabilistycznym.

